

AW

Aplicaciones web

01
UNIDAD
01
CLASE

HTML



| HTML5

| Comprender el perfil de un Desarrollador y un diseñador

| Los alumnos comprenderán la diferencia y la importancia entre los lenguajes estructurados y dinámicos, compilados e interpretados.



| Comprender porque se estudia HTML5 en la materia y la importancia de este en el ámbito laboral

| Conocer y comprender la evolución, versiones y características del lenguaje HTML5.

ISSD

-Des-

Desarrollo de
Software

**MÓDULO
DIDÁCTICO**



Introducción

En la actualidad todos quieren construir un sitio web, ya sea para fomentar su negocio o servicio online o diseñar un sitio para algún familiar o amigo, y la mayor pregunta entra en juego: *¿Cómo lo hago?, ¿Por dónde comienzo, ¿Será complicado aprender a diseñar y programar un sitio?, ¿Cuál será la salida laboral que tendré una vez egresado?* Estas y otras preguntas posiblemente invadan tu cabeza al momento de plantearte tu futuro.

Aprender cualquier lenguaje de programación web, te dará las respuestas que estás buscando. Deberás comenzar por HTML el que te ofrecerá las posibilidades de expandir tu mente y elevar tu coeficiente intelectual. Es un buen punto para comenzar si es que intentas aprender más allá del mismo. Codificar en HTML5 y CSS3 te brindará práctica mientras que, siendo un lenguaje simple, te introducirá al mundo de la programación (también a parte del diseño y desarrollo web). Te deseo lo mejor en el nuevo camino que elegiste para tu futuro, no te arrepentirás!!!



Desempeño de Exploración

Desde la intuición y la propia experiencia de usuario de internet, conversamos y debatimos en clase los nuevos conceptos adquiridos por otras materias y la experiencia que trae el alumno con respecto a los tópicos de la materia y meta de comprensión. También se hace hincapié en las expectativas laborales y profesionales que poseen los alumnos y lo que los impulsa o motiva a cursar esta carrera.

Introducción a la Clase

En esta clase, solo presentaremos conceptos generales que debes saber antes de comenzar a programar en HTML, estos conceptos te acompañaran a lo largo de la carrera siendo profundizados en cada uno de los semestre. Espero que te agrade el materia y que la lectura te sea amena. No dejes de consultar ante cualquier duda, inquietud que se presente en tu camino como también buscar información adicional sobre los temas dados esto te ayudará a incrementar tus conocimientos y habilidad en los tópicos.

Suerte en la clase!!

Páginas Web

Podemos determinar conceptualmente que un **sitio Web** es un conjunto de archivos, en donde cada archivo está íntimamente relacionado con el otro a través de links (uniones entre páginas). Los archivos tienen que seguir una serie de reglas para que puedan ser visualizados por cualquier navegador existente, por esto se creó un lenguaje llamado **HTML**, que consta de un sin número de *etiquetas* dentro de las cuales se pueden definir, qué, y cómo se mostrará cada elemento en la Web.

Siguiendo este concepto podemos decir que una página Web es un conjunto de etiquetas perfectamente organizadas en archivos y que permiten reflejar la información de un tema a tratar.

Los archivos **HTML** son archivos de textos en donde su creador incorpora las etiquetas siguiendo un orden propio del lenguaje.

Lenguajes de programación

Un lenguaje de programación es un lenguaje que puede ser utilizado para controlar el comportamiento de una máquina, particularmente una computadora. Consiste en un conjunto de símbolos y reglas sintácticas y semánticas que definen su estructura y el significado de sus elementos y expresiones.

Los procesadores usados en las computadoras son capaces de entender y actuar según lo indican programas escritos en un lenguaje fijo llamado **lenguaje de máquina**. Todo programa escrito en otro lenguaje puede ser ejecutado de dos maneras:

1| Mediante un programa que va adaptando las instrucciones conforme son encontradas. A este proceso se lo llama **interpretar** y a los programas que lo hacen se los conoce como **intérpretes**.

2| Traduciendo este programa al programa equivalente escrito en lenguaje de máquina. A ese proceso se lo llama **compilar** y al traductor se lo conoce como un mal hecho **compilador**.

Los lenguajes de programación se determinan según el nivel de abstracción, Según la forma de ejecución y Según el paradigma de programación que poseen cada uno de ellos y esos pueden ser:

Según su nivel de abstracción

| Lenguajes de bajo nivel: Los lenguajes de bajo nivel son lenguajes de programación que se acercan al funcionamiento de una computadora. El lenguaje de más bajo nivel es, por excelencia, **el código máquina**. A éste le sigue el lenguaje **ensamblador**, ya que al programar en ensamblador se trabajan con los registros de memoria de la computadora de forma directa.

| Lenguajes de medio nivel: Hay lenguajes de programación que son considerados por algunos expertos como lenguajes de medio nivel (como es el caso del lenguaje C) al tener ciertas características que los acercan a los lenguajes de bajo nivel pero teniendo, al mismo tiempo, ciertas cualidades que lo hacen un lenguaje más cercano al humano y, por tanto, de alto nivel.

| Lenguajes de alto nivel: Los lenguajes de alto nivel son normalmente fáciles de aprender porque están formados por elementos de lenguajes naturales, como el inglés.

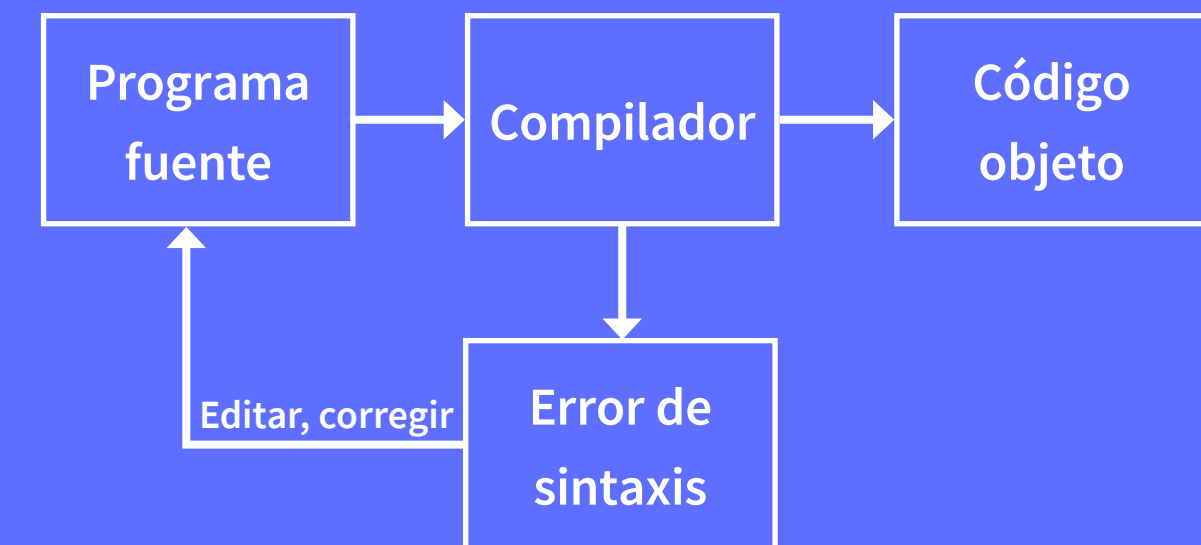
Según la forma de ejecución

| **Lenguajes compilados:** Naturalmente, un programa que se escribe en un lenguaje de alto nivel también tiene que traducirse a un código que pueda utilizar la máquina. Los programas traductores que pueden realizar esta operación se llaman **compiladores**. Éstos, como los programas ensambladores avanzados, pueden generar muchas líneas de código de máquina por cada proposición del programa fuente. Se requiere una corrida de compilación antes de procesar los datos de un problema. Los compiladores son aquellos cuya función es traducir un programa escrito en un determinado lenguaje a un idioma que la computadora entienda (lenguaje máquina con código binario). Al usar un lenguaje compilado (como lo son los lenguajes del popular Visual Studio de Microsoft), el programa desarrollado nunca se ejecuta mientras haya errores, sino hasta que luego de haber compilado el programa, ya no aparecen errores en el código.

Figura 1

| **Lenguajes interpretados:** Se puede también utilizar una alternativa diferente de los compiladores para traducir lenguajes de alto nivel. En vez de traducir el programa fuente y grabar en forma permanente el código objeto que se produce durante la corrida de compilación para utilizarlo en una corrida de producción futura, el programador sólo carga el programa fuente en la computadora junto con los datos que se van a procesar. A continuación, un programa intérprete, almacenado en el sistema operativo del disco, o incluido de manera permanente dentro de la máquina, convierte cada proposición del programa fuente en lenguaje de máquina conforme vaya siendo necesario durante el proceso de los datos. No se graba el código objeto para utilizarlo posteriormente.

Figura 1



La siguiente vez que se utilice una instrucción, se le debe interpretar otra vez y traducir a lenguaje máquina. Por ejemplo, durante el procesamiento repetitivo de los pasos de un ciclo, cada instrucción del ciclo tendrá que volver a ser interpretado cada vez que se ejecute el ciclo, lo cual hace que el programa sea **más lento en tiempo de ejecución** (porque se va revisando el código en tiempo de ejecución) pero **más rápido en tiempo de diseño** (porque no se tiene que estar compilando a cada momento el código completo). El intérprete elimina la necesidad de realizar una corrida de compilación después de cada modificación del programa cuando se quiere agregar funciones o corregir errores; pero es obvio que un programa objeto compilado con antelación deberá ejecutarse con mucha mayor rapidez que uno que se debe interpretar a cada paso durante una corrida de producción.

Según el paradigma de programación

Un **paradigma de programación** representa un enfoque particular o filosofía para la construcción del software. No es mejor uno que otro, sino que cada uno tiene ventajas y desventajas. Dependiendo de la situación un paradigma resulta más apropiado que otro. Atendiendo al paradigma de programación, se pueden clasificar los lenguajes en:

| **El paradigma imperativo o por procedimientos** es considerado el más común y está representado, por ejemplo, por el **C** o por **BASIC**. El paradigma funcional está representado por la familia de lenguajes **LISP** (en particular Scheme).

| **El paradigma lógico**, un ejemplo es **PROLOG**.

| **El paradigma orientado a objetos**. Un lenguaje completamente orientado a objetos es **Smalltalk**.

Al hablar de Lenguajes de Programación es obligado tocar el tema de **Programación**. En informática la programación es un proceso por el cual se escribe (en un lenguaje de programación), se prueba, se depura y se mantiene el código fuente de un programa informático. Dentro de la informática, los programas son los elementos que forman el software, que es el **conjunto de las instrucciones que ejecuta el hardware** de una computadora **para realizar una tarea determinada**.

Lenguajes para la web

Actualmente existen diferentes lenguajes de programación para desarrollar en la web, estos han ido surgiendo debido a las tendencias y necesidades de las plataformas. En el presente artículo pretende mostrar las ventajas y desventajas de los lenguajes más conocidos.

Desde los inicios de Internet, fueron surgiendo diferentes demandas por los usuarios y se dieron soluciones mediante lenguajes estáticos. A medida que paso el tiempo, las tecnologías fueron desarrollándose y surgieron nuevos problemas a dar solución. Esto dio lugar a desarrollar lenguajes de programación para la web dinámica, que permitieran interactuar con los usuarios y utilizaran sistemas de Bases de Datos. A continuación, veremos algunos lenguajes con los cuales podremos programar para la web:

| **HTML:** Desde el surgimiento de internet se han publicado sitios web gracias al lenguaje HTML. Es un lenguaje estático para el **desarrollo de sitios web** (acrónimo en inglés de HyperText Markup Language, -Lenguaje de Marcas Hipertextuales). Desarrollado por el *World Wide Web Consortium* (W3C). Los archivos pueden tener las extensiones (htm, html).

Ventajas:

- | Texto presentado de forma estructurada y agradable.
- | No necesita de grandes conocimientos cuando se cuenta con un editor de páginas web o WYSIWYG.
- | Archivos pequeños.
- | Despliegue rápido.
- | Lenguaje de fácil aprendizaje.
- | Lo admiten todos los exploradores.

Desventajas:

- | Lenguaje estático.
- | La interpretación de cada navegador puede ser diferente.
- | El diseño es más lento.
- | Las etiquetas son muy limitadas.

| **Javascript:** Este es un lenguaje interpretado, no requiere compilación. Fue creado por *Brendan Eich* en la empresa *Netscape Communications*. Utilizado principalmente en **páginas web**. Es similar a Java, aunque no es un lenguaje orientado a objetos, el mismo no dispone de herencias. La mayoría de los navegadores en sus últimas versiones interpretan código JavaScript

Ventajas:

- | Lenguaje de scripting seguro y fiable.
- | Los script tienen capacidades limitadas, por razones de seguridad.
- | El código Javascript se ejecuta en el cliente.

Desventajas:

- | Código visible por cualquier usuario.
- | El código debe descargarse completamente.
- | Puede poner en riesgo la seguridad del sitio, con el actual problema llamado XSS (significa en inglés Cross Site Scripting renombrado a XSS por su similitud con las hojas de estilo CSS).

| **PHP:** Es un lenguaje de programación utilizado para la creación de sitio web. PHP es un acrónimo recursivo que significa “*PHP Hy-pertext Preprocessor*”, (inicialmente se llamó Personal Home Page). Surgió en 1995, desarrollado por *PHP Group*.

PHP es un lenguaje de script interpretado en el lado del servidor utilizado para la generación de **páginas web dinámicas**, embebidas en páginas HTML y ejecutadas en el servidor. PHP no necesita ser compilado para ejecutarse. Para su funcionamiento necesita tener instalado **Apache** o **IIS** con las librerías de PHP. La mayor parte de su sintaxis ha sido tomada de C, Java y Perl con algunas características específicas. Los archivos cuentan con la extensión (php).

Ventajas:

- | Se caracteriza por ser un lenguaje muy rápido.
- | Soporta en cierta medida la orientación a objeto. Clases y herencia.
- | Es un lenguaje multiplataforma: Linux, Windows, entre otros.
- | Capacidad de conexión con la mayoría de los manejadores de base de datos: MySQL, PostgreSQL, Oracle, MS SQL Server, entre otras.
- | Capacidad de expandir su potencial utilizando módulos.
- | Es libre, por lo que se presenta como una alternativa de fácil acceso para todos.
- | Incluye gran cantidad de funciones.
- | No requiere definición de tipos de variables ni manejo detallado del bajo nivel.

Desventajas:

- | Se necesita instalar un servidor web.
- | Todo el trabajo lo realiza el servidor y no delega al cliente. Por tanto puede ser más ineficiente a medida que las solicitudes aumenten de número.
- | La legibilidad del código puede verse afectada al mezclar sentencias HTML y PHP.
- | La programación orientada a objetos es aún muy deficiente para aplicaciones grandes.
- | Dificulta la modularización.
- | Dificulta la organización por capas de la aplicación.

| **ASP:** Es una tecnología del lado de servidor desarrollada por *Microsoft* para el desarrollo de **sitio web dinámicos**. ASP significa en inglés (Active Server Pages), fue liberado por Microsoft en 1996. Las páginas web desarrolladas bajo este lenguaje es necesario tener instalado **Intertarse**. Existen varios lenguajes que se pueden utilizar para crear páginas ASP. El más utilizado es **VBScript**, nativo de Microsoft. ASP se puede hacer también en **Perl and Jscript** (no JavaScript). El código ASP puede ser insertado junto con el código HTML. Los archivos cuentan con la extensión (asp).

Ventajas:

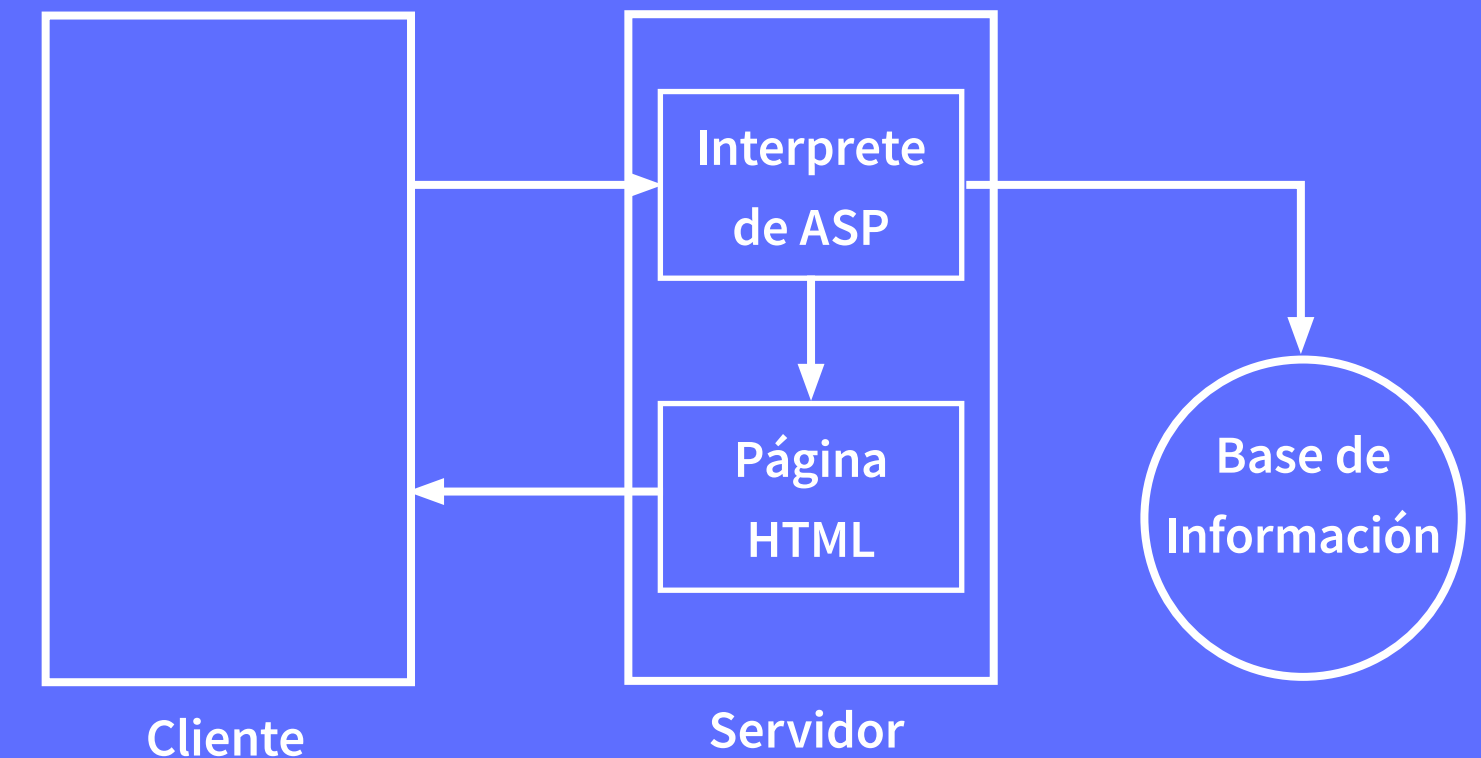
- | Usa Visual Basic Script, siendo fácil para los usuarios.
- | Comunicación óptima con SQL Server.
- | Soporta el lenguaje JScript (Javascript de Microsoft).

Desventajas:

- | Código desorganizado.
- | Se necesita escribir mucho código para realizar funciones sencillas.
- | Tecnología propietaria.
- | Hospedaje de sitios web costosos.

Figura 2

Figura 2 Esquema de funcionamiento de ASP



| **ASP.NET:** Este es un lenguaje comercializado por *Microsoft*, y *usaASP.NET* es el sucesor de la tecnología ASP, fue lanzada al mercado mediante una estrategia de mercado denominada **.NET**. ASP.NET fue desarrollado para resolver las limitantes que brindaba tu antecesor ASP. Creado para **desarrollar web sencillas o grandes aplicaciones**. Para el desarrollo de ASP.NET se puede utilizar **C#, VB.NET** o **J#**. Los archivos cuentan con la extensión (aspx). Para su funcionamiento de las páginas se necesita tener instalado **IIS** con el Framework .Net. Microsoft Windows 2003 incluye este framework, solo se necesitará instalarlo en versiones anteriores.

Ventajas:

- | Completamente orientado a objetos.
- Controles de usuario y personalizados
- | División entre la capa de aplicación o diseño y el código.
- | Facilita el mantenimiento de grandes aplicaciones.
- | Incremento de velocidad de respuesta del servidor.
- | Mayor velocidad y seguridad

Desventajas:

- | Consumo de recursos.

| **JSP:** Es un lenguaje para la creación de sitios web dinámicos, acrónimo de Java Server Pages. Está orientado a desarrollar **páginas web en Java**. JSP es un lenguaje multiplataforma. Creado para ejecutarse del lado del servidor. JSP fue desarrollado por *Sun Microsystems*. Comparte ventajas similares a las de ASP.NET, desarrollado para la creación de aplicaciones web potentes. Posee un motor de páginas basado en los **servlets** de Java. Para su funcionamiento se necesita tener instalado un servidor **Tomcat**.

Características:

- | Código separado de la lógica del programa.
- | Las páginas son compiladas en la primera petición.
- | Permite separar la parte dinámica de la estática en las páginas web.
- | Los archivos se encuentran con la extensión (jsp).
- | El código JSP puede ser incrustado en código HTML.

Los elementos que pueden ser insertados en las páginas JSP son los siguientes:

- | Código: se puede incrustar código “Java”.
- | Directivas: permite controlar parámetros del servlet.
- | Acciones: permite alterar el flujo normal de ejecución de una página.

Ventajas:

- | Ejecución rápida del servlets.
- | Crear páginas del lado del servidor.
- | Multiplataforma.
- | Código bien estructurado.
- | Integridad con los módulos de Java.
- | La parte dinámica está escrita en Java.
- | Permite la utilización de servlets.

Glosario

servlets: son objetos que corren dentro del contexto de un contenedor de servlets (ej: Tomcat) y extienden su funcionalidad.

Programa Java del lado del servi-dor que ofrece funciones suplementarias al servidor.

Aplicación JAVA que permite la ejecución de un propio servidor web que permite la interactividad del usuario, permitiéndole realizar algunas opciones. Es diferente a CGI.

Desventajas:

- | Complejidad de aprendizaje.

| **Python:** Es un lenguaje de programación creado en el año 1990 por *Guido van Rossum*, es el sucesor del lenguaje de programación ABC. Python es comparado habitualmente con Perl. Los usuarios lo consideran como un lenguaje más limpio para programar. Permite la creación de todo tipo de programas incluyendo los sitios web. Su código no necesita ser compilado, por lo que se llama que el código es interpretado. Es un lenguaje de programación multiparadigma, lo cual fuerza a que los programadores adopten por un estilo de programación particular:

- | Programación orientada a objetos.
- | Programación estructurada.
- | Programación funcional.
- | Programación orientada a aspectos.

Ventajas:

- | Libre y fuente abierta.
- | Lenguaje de propósito general.
- | Gran cantidad de funciones y librerías.
- | Sencillo y rápido de programar.
- | Multiplataforma.
- | Licencia de código abierto (Opensource).
- | Orientado a Objetos.
- | Portable.

Desventajas:

- | Lentitud por ser un lenguaje interpretado.

| **Ruby:** Es un lenguaje interpretado de muy alto nivel y orientado a objetos. Desarrollado en el 1993 por el programador japonés *Yukihiro “Matz” Matsumoto*. Su sintaxis está inspirada en Python, Perl. Es distribuido bajo licencia de software libre (OpenSource). Ruby es un lenguaje dinámico para una **programación orientada a objetos rápida y sencilla**. Para los que deseen iniciarse en este lenguaje pueden encontrar un tutorial interactivo de ruby.

Características:

- | Existe diferencia entre mayúsculas y minúsculas.
- | Múltiples expresiones por líneas, separadas por punto y coma “;”.
- | Dispone de manejo de excepciones.
- | Ruby puede cargar librerías de extensiones dinámicamente si el (Sistema Operativo) lo permite.
- | Portátil.

Ventajas:

- | Permite desarrollar soluciones a bajo Costo.
- | Software libre.
- | Multiplataforma.

| **ColdFusion:** es un entorno de desarrollo Web dinámico y un servidor Web que permite trabajar con distintos lenguajes como ASP, PHP, JSP, etc. Integra el **ColdFusion Markup Language** un lenguaje creado por *Macromedia* (ahora Adobe) cuyo funcionamiento se basa en etiquetas especiales integradas sobre el código HTML. Es una plataforma que se puede ejecutar de forma concurrente con la mayoría de los servidores Web de Windows, Mac, Linux y Solaris.

Lenguajes Dinámicos Vs Lenguajes Estático

Antes de que se popularizaran los lenguajes de desarrollo Web, la forma clásica de realizar un sitio Web consistía en escribir las páginas directamente con código HTML a través de un editor Web. Esta tarea es factible cuando se trata de sitios con muy poco contenido y que no se actualizan con frecuencia, pero se convierte en desesperante en aquellos sitios con muchos contenidos y que incorporan novedades con asiduidad. Por ejemplo, si se quieren realizar en HTML cambios sobre algún elemento común a todas las páginas del sitio, se deben aplicar en todas las páginas, una por una, con lo que se convierte en un trabajo muy tedioso. Los lenguajes de desarrollo Web intentan facilitar las tareas de los creadores de aplicaciones, de manera que se **automaticen los procesos** y se **multipliquen las posibilidades**.

De este modo, mientras mediante HTML *sólo es posible crear sitios Web estáticos*, mediante los lenguajes de desarrollo Web se pueden crear sitios Web dinámicos. Un sitio Web dinámico es uno que puede tener **cambios frecuentes en la información**. Cuando el servidor Web recibe una petición para una determinada página de un sitio Web, la página se genera automáticamente por el software como respuesta directa a la petición de la página. Es decir, una página Web dinámica es una página que permite al usuario interactuar con ella, que permite actualizar los datos ofrecidos sin tener que ser editada de nuevo y que contiene efectos especiales.

Para crear una página dinámica no basta con programar en HTML, ya que este lenguaje, como veremos, es muy limitado. Es necesario combinar HTML con otros lenguajes, como Perl, PHP, JSP, ASP.NET, JavaScript, Java, etc. **La generación del contenido dinámico puede suceder en el servidor o en el cliente**, empleándose por lo general lenguajes distintos en cada caso, si bien hay lenguajes que pueden trabajar según ambos paradigmas. Cada lenguaje tiene unas reglas de programación y un funcionamiento distinto. A la combinación de estos elementos se le conoce como **HTML dinámico** o **DHTML** (Dynamic HTML).

Claves para un sitio web dinámico

| **Interactividad.** Un sitio web dinámico debe permitir al visitante encontrar su propio camino hasta la información que está buscando, en lugar de sentarse pasivamente recibiendo información que está buscando en lugar de sentarse pasivamente recibiendo información Tomando decisiones y realizando tareas los visitantes e involucra en múltiples niveles creando así una experiencia más robusta.

| **Sincronización.** Un sitio web dinámico debe **reunir la información y actividades** relevantes de **distintas fuentes** ya sea directamente o a través de vínculos, en el momento en que el visitante los necesita. Un ejemplo es cuando un visitante escoge en una lista con las ciudades de ese país.

| **Flexibilidad.** Un sitio web dinámico debe proporcionar al visitante **distintas formas** de encontrar la información o realizar tareas para que pueda escoger **el método que mejor se ajuste a sus necesidades**. Por ejemplo podemos presentarle menús y campos de búsqueda para que navegue por el sitio, permitiendo al visitante que escoja lo que mejor le venga.

| **Adaptabilidad.** Un sitio web dinámico se ajusta para ofrecer servicio a las necesidades de cada visitante individual. A veces este ajuste se hace en el servidor a través de personalización de contenido pero los diseñadores de web pueden hacer mucho más para acomodar al visitante sin requerir que este cargue una nueva página Web. Por ejemplo, los menús que se deslizan dentro y fuera de uno de los lados de la ventana siempre están disponibles para visitantes que se están moviendo constantemente, pero pueden ocultarse para maximizar el área de visualización.

| **Actividad.** Un sitio Web dinámico utiliza movimientos y sonido para atraer la atención sobre los cambios en pantalla Presentada a menudo en la forma de animación, **la acción** es uno de los principios dinámicos más difíciles de aplicar con eficacia. La actividad puede ser parte integral de la experiencia dinámica si se utiliza para recalcar la interactividad, por ejemplo, si se usa una animación en el estado de paso del ratón de un vínculo para crear un efecto más pronunciado.

En resumen te recordamos que una **página web estática** es una página web donde el contenido, el HTML y los gráficos, son siempre estáticos se sirve a cualquier visitante de la misma manera, a no ser que la persona que ha creado la web decida cambiar manualmente su copia en el servidor, exactamente lo que hemos estado repasando en la mayor parte de este apartado.

Por el contrario, en una **página web dinámica**, el contenido del servidor es el mismo, pero en vez de ser sólo HTML, también contiene código dinámico, que puede mostrar datos diferentes según la información que suministre a la página web.

Otra cosa que cabe tener en cuenta es que se debe instalar un software especial en el servidor para crear una página web dinámica. Mientras que los ficheros HTML estáticos normales se guardan con una extensión de fichero .html, estos ficheros contienen código dinámico especial además del HTML y se guardan con extensiones de archivo especiales para indicarle al servidor web que necesitan un procesamiento adicional antes de enviarlos al cliente (como, por ejemplo, que se inserten los datos desde la base de datos); los archivos PHP, por ejemplo, generalmente tienen una extensión de archivo .php.

Hay muchos lenguajes dinámicos que se pueden elegir: el PHP que hemos mencionado antes y otros como Python, Ruby, ASP.NET y Coldfusion. En definitiva, todos estos lenguajes tienen más o menos las mismas capacidades, como hablar con bases de datos, validar la información introducida en los formularios, etc., pero hacen las cosas de manera ligeramente diferente y tienen algunas ventajas e inconvenientes. Todo se reduce a la forma más sencilla que mejor se adapte.

¿Por qué aprender HTML?

Una de las preguntas más frecuentes en los foros para principiantes es “¿Cómo iniciarse en programación?” o “¿Qué lenguaje de programación aprender primero?”. La segunda sería; ¿Debo aprender a escribir código HTML manualmente o usar un editor WYSIWYG (lo que ves es lo que obtienes ejemplo: Dreaweaver)? La respuesta a dicha pregunta depende del objetivo que cada uno tiene. Escribir código HTML manualmente tiene sus ventajas y desventajas así como los editores WYSIWYG. Para saber cuál de ellos es mejor para ti, deberás conocer sus ventajas y desventajas y sacar una conclusión basada en ellas y en tus intenciones una vez que hayas cursado algunos semestres.

Para comenzar, no existe un lenguaje de programación para iniciarse en programación, la programación se comienza sobre un papel, haciendo algoritmos. Claro que para ver si los algoritmos funcionan puede ser más cómodo utilizar un lenguaje de programación. En este caso lo único que recomiendo es utilizar un lenguaje simple que no sea orientado a objetos, ni gráfico.

Esto posiblemente suceda debido a que la respuesta a las preguntas “¿debería aprender código HTML o simplemente usar un editor WYSIWYG? ¿Qué es mejor para mí?” depende en mayor medida de tus expectativas, necesidades e intenciones y no solo en las características de las herramientas disponibles. Mientras algunas personas prefieren medir cosas utilizando una escuadra debido a que

Glosario

WYSIWYG: es el acrónimo de What You See Is What You Get (en inglés, "lo que ves es lo que es"). Filosofía de programación y maquetación, cuya finalidad quiere decir, que lo que tú ves mientras editas, es lo que vas a obtener como producto final.

poseen más usos, otras prefieren la regla ya que es más portable. Cada objeto tiene sus ventajas y desventajas y no se puede afirmar con exactitud cuál de ellos es mejor. De modo que para aclarar este punto echemos un vistazo a las diferencias entre **HTML** y los editores **WYSIWYG**.

HTML (HyperText Mark-Up Language) es lo que se conoce como "**lenguaje de marcado**", cuya función es preparar documentos escritos aplicando etiquetas de formato. Las etiquetas indican cómo se **presenta el documento** y cómo **se vincula a otros documentos**.

HTML se usa también para la lectura de documentos en Internet desde diferentes equipos gracias al **protocolo HTTP**, que permite a los usuarios acceder, de forma remota, a documentos almacenados en una dirección específica de la red, denominada dirección URL.

La World Wide Web (WWW), o simplemente la Web, es la red mundial formada por todos los documentos (llamados "páginas Web") **conectados entre sí por hipervínculos**.

A menudo, las páginas web se organizan alrededor de una página principal que actúa como eje central para buscar otras páginas con hipervínculos. Este grupo de páginas Web unidas por hipervínculos y centradas alrededor de una página principal se llama **sitio Web**.

La Web es un amplio archivo dinámico compuesto de una gran variedad de sitios Web y que permite el acceso a páginas Web que contienen texto formateado, imágenes, sonidos, videos, etc.

Cuál es la función de un Maquetador?

El **maquetador** debe convertir un diseño hecho por los diseñadores, normalmente varios ficheros Photoshop, en páginas estáticas **XHTML/CSS** con las imágenes recortadas en formatos adaptados para la web, para que luego sean programadas por los desarrolladores web.

Funciones de un Desarrollador web

Podríamos ver al perfil de **desarrollador web** como un contenedor de perfiles... un desarrollador web hace de todo, y muy bien: maqueta, programa en cliente y en servidor, e incluso puede hacer diseños, dependiendo de la capacidad artística de cada uno. No basta con que sepa un poco de HTML y un poco de PHP. Es decir, como el mis-mo nombre induce a entender, un desarrollador web es un experto en desarrollo web, y debe dominar todas las tecnologías que convergen en una web.

Navegadores de la web

Un navegador o navegador web (del inglés, web browser) es un programa que permite ver la información que contiene una página web (ya se encuentre ésta alojada en un servidor dentro de la World Wide Web o en un servidor local).

El navegador **interpreta el código**, HTML generalmente, en el que está escrita la página web y lo presenta en pantalla permitiendo al usuario **interactuar** con su contenido y **navegar** hacia otros lugares de la red mediante enlaces o hipervínculos.

La funcionalidad básica de un navegador web es permitir la **visualización de documentos de texto**, posiblemente con recursos multimedia incrustados. Los documentos pueden estar ubicados en la computadora en donde está el usuario, pero también pueden estar en cualquier otro dispositivo que esté conectado a la computadora del usuario o a través de Internet, y que tenga los recursos necesarios para la transmisión de los documentos (un software servidor web).

Tales documentos, comúnmente denominados páginas web, poseen **hipervínculos** que enlazan una porción de texto o una imagen a otro documento, normalmente relacionado con el texto o la imagen.

El seguimiento de enlaces de una página a otra, ubicada en cualquier computadora conectada a la Internet, se llama **navegación**, de donde se origina el nombre navegador (aplicado tanto para el programa como para la persona que lo utiliza, a la cual también se le llama cibernauta).

Historia y Evolución

Como ya se dijo anteriormente HTML nació en 1980 como un proyecto de *Tim Berners-Lee* basado en el concepto de **hipertexto**, que ayudaría a investigadores a compartir información en forma de documentos sobre Internet. Fue implementado más tarde en 1989 en la **CERN** (organización europea para la investigación nuclear), el nodo más grande en Europa. Desde allí, HTML comenzó su evolución que no está aún concluida, pasando por las versiones 2.0, 3.2, 4.0 y 4.01, todas ellas basadas en **SGML** (lenguaje de etiquetado estándar generalizado: un metalenguaje usado para crear otros lenguajes como sublenguajes del mismo).

Por otro lado, **XML** (lenguaje de marcas extensible) es también un metalenguaje (usado para crear otros lenguajes) y es también un sub lenguaje de, diseñado para ser más simple de procesar. En estos días, XML es ampliamente utilizado en diferentes formas para construir documentos y organizar información (por ejemplo, RSS (redifusión realmente simple), Atom, etc.) ya que provee una forma estándar de lograrlo que es más fácil de procesar que **SGML**.

En el año 2000, **XHTML** es recomendado por el World Wide Web Consortium (W3C) como la **nueva versión estándar** de HTML basada en **XML** en lugar de **SGML**. De esta forma, podemos considerar a **XHTML** como el resultado de mezclar **HTML** y **XML**. Hecho esto, todos los beneficios de **XML** son ahora heredados por HTML lo que lo hace más fácil de procesar, y por lo tanto estar disponible en más plataformas con capacidades de procesamiento reducidas (por ejemplo, PDAs (asistente digital personal) y teléfonos celulares).

Otro motivo para actualizar las versiones de **HTML** y para la creación del **W3C** es el reestablecimiento del propósito original de **HTML** como un lenguaje semántico. Desde que fue implementado, muchos fabricantes de navegadores comenzaron a transformar el estándar con el objeto de agregarle más funcionalidad. Esto lo convirtió lentamente en un lenguaje **más visual que semántico**, lo que inspiró al W3C a crear nuevos estándares pensados para revertir este efecto y retornarlo a su origen semántico. **XHTML 1.1** es la más reciente de estas actualizaciones pero hay más por venir.

El HTML fue desarrollado originalmente por *Tim Bernes-Lee* en el **CERN** (Centro Europeo de Física de Partículas), y fue popularizado por el navegador Mosaic desarrollado por la **NCSA** (National Center for Supercomputing Applications). Pero debido al rápido crecimiento de la web, surgió la necesidad de crear un estándar para que tanto los autores como los navegadores pudieran **reconocer cualquier versión de HTML**.

Los estándares HTML son el **HTML 2.0**, el **HTML 3.2**, el **HTML 4.0** y el **HTML 4.01**., **HTML 5**.

Versiones

| **HTML 1.0** Publicado en 1991 con el nombre HTML Tags.

| **HTML 2.0** El 22 de septiembre de 1995 se publica esta versión, que es la primera versión estándar.

| **HTML 3.2** Versión publicada el 14 de enero de 1997 que incorpora los últimos avances en diseño de páginas web, texto que se podía colocar alrededor de las imágenes y applets de Java.

| **HTML 4.0** Fue una versión corregida de la original publicada en diciembre de 1997. Se publicó en abril de 1998. Ahora se pueden usar las hojas de estilo CSS, que antes no se usaban, incluir scripts en las páginas web, entre otras mejoras.

| **HTML 4.01** Publicada en diciembre de 1999, que es una revisión y actualización de la versión anterior.

| **HTML 5.0** El 22 de enero de 2008 se publicó un borrador de esta versión y se está en la espera de la publicación oficial.

Características de HTML 5

HTML, “Hypertext Markup Language” o como diríamos en español: **Lenguaje marcado de hipertexto** llega a su versión 5 con cambios. Según comentan en el W3C y en un fenomenal artículo de IBM, **HTML5** incluirá nuevos elementos por primera vez en el último milenio.

Figura 3

| Una nueva estrategia de **tagging**. En vez de usar el elemento object o embed para todo, el audio y el video tendrán su propio tag

| **Bases de datos localizadas**. Esta funcionalidad, cuando esté implementada, embeberá una base de datos SQL local la cual podrá ser accedida por el sitio web.

Glosario

GPS: (Global Positioning System: sistema de posicionamiento global) o NAVSTAR-GPS es un sistema global de navegación por satélite (GNSS) que permite determinar en todo el mundo la posición de un objeto, una persona, un vehículo o una nave, con una precisión hasta de centímetros (si se utiliza GPS diferencial), aunque lo habitual son unos pocos metros de precisión. El sistema fue desarrollado, instalado y actualmente operado por el Departamento de Defensa de los Estados Unidos.

Figura 3

HTML5 ≈ HTML + CSS + JS APIs

| Gracias al elemento **canvas**, será posible realizar animaciones vectoriales sin el uso de plugins como silverlight o flash.

| **Aplicaciones reales en los browsers.** Será posible crear, mediante APIs, aplicaciones de escritorio “reales” con capacidades como drag and drop, entre otras

| **Los tags** que quedan sobre la presentación serán definitivamente eliminados y el CSS será el que se encargue de la presentación

| Otra de las nuevas características de **HTML5** es el soporte para **geolocalización**, es decir, para poder posicionar al usuario en el lugar donde se encuentre y compartir esta información, si fuese necesario. En la actualidad, la posición de los usuarios se localiza mediante la dirección IP de su conexión, los satélites **GPS** a los que accede su móvil o mediante los datos de la torre de comunicaciones a la cual estén conectados, en el caso de utilizar Internet en el móvil. Con esta nueva etiqueta en HTML5, se podrán conocer en todo momento estos datos con exactitud y aunque el usuario varíe su posición.

El logo está representado con ocho iconos representativos de las tecnologías que forman parte de HTML5. De **izquierda a derecha** podemos leer:



| **Semántica:** se utilizan etiquetas enriquecidas, RDFa, microformatos...

| **Offline y Almacenamiento:** no se requiere conexión a internet y se utiliza App Cache, Local Storage, Indexed DB, o la API de File.

| **Acceso al Dispositivo:** la aplicación web interactúa con el dispositivo para obtener información, por ejemplo, como la geolocalización.

| **Conectividad:** se utiliza Web Sockets y gestión de eventos al servidor para mejorar la comunicación y ofrecer chats en tiempo real, actualizaciones push...

| **Multimedia:** se utilizan las nuevas etiquetas de audio y vídeo de HTML5.

| **3D, Gráficos y Efectos:** se utiliza SVG, Canvas, WebGL, o CSS3 3D para ofrecer una interfaz visual más atractiva renderizada de forma nativa por el navegador.

| **Optimización e Integración:** se utilizan tecnologías como Web Workers o XMLHttpRequest 2 para ofrecer al usuario una web más rápida y sin esperas de carga.

| **CSS3:** se utilizan las últimas características de estilo mejorando el diseño de las aplicaciones. También se incluyen tecnologías co-mo Web Open Font Format (WOFF) para controlar las tipografías utilizadas en la web.

HTML5 viene a suplir al ya decadente HTML4, siendo recomendado para construir los proyectos venideros, pues en su combinación con otros lenguajes, puede brindarte excelentes oportunidades para explotar al máximo las nuevas páginas web, pero utilizando menos recursos. Además. Con la implementación de HTML5, el formato **XHTML** quedaría sin ser necesario.

Por varios motivos, cada desarrollador de sitios web debe conocer las ventajas de HTML5, pues muchas de las grandes compañías están volcándose a su implementación. Por una parte, Google ha estado trabajando este formato con Chrome desde hace algunos años, como también Adobe que, desde hace algún tiempo removi6 el soporte de Flash para Android, dando paso a la llegada de HTML5.

Esencialmente, HTML5 es un sistema para **formatear el layout de tus páginas web**, así como para ajustar el aspecto de las mismas. Utilizando HTML5, algunos navegadores como Chrome, Firefox, Explorer, Safari y otros, pueden conocer la forma de cómo proyectar determinada página web, donde ubicar los elementos, donde colocar las imágenes o el texto y más. La diferencia principal de HTML5 con respecto a sus versiones anteriores, es el nivel de sofisticación del código que puedes desarrollar utilizando este formato.

Hablando de **Markup**, HTML5 añade varios elementos para actualizarse a los tiempos modernos que transcurren. Muchas de sus novedades, tienen relación a la forma de crear sitios web que actualmente puedes observar, una de sus novedades más interesantes está relacionada con la **inserción de multimedia en los websites**, mismos que podrán contar con etiquetas HTML especiales para tener opción a ser añadidos. Así mismo, hay aspectos en diseño que también son agregados al lenguaje, de igual forma a ciertos detalles en navegación.

Si utilizas HTML5 para crear una web, puedes reducir considerablemente la dependencia de los **plug-ins** que los usuarios deberían tener instalados para poder observar correctamente tu sitio web. Como en el caso de Adobe Flash que sí se ve afectado por la implementación de este nuevo estándar, pero que surge como una excelente alternativa para dispositivos que en forma nativa no soportaban Flash u otro tipo de plug-ins necesarios. Además, HTML5 amplía el campo para el desarrollo de aplicaciones que puedan ser utilizadas en múltiples clases de dispositivos.

Con la implementación de HTML5, puedes acceder a los sitios web de forma **off-line**. También se añade la funcionalidad de **drag and drop** y la edición de documentos de **forma online**, misma que fuese popularizada con Google Docs. Otro de sus puntos fuertes es la **geolocalización**, aunque también están las etiquetas especialmente diseñadas para ahorrar la necesidad de plug-ins de Flash para reproducir audio o vídeo en las webs. Con esto, Flash recibe un duro golpe que, a pesar de seguir siendo utilizado, HTML5 aún no ha dado el gran salto.

Las reglas básicas que se plantearon a la hora de actualizar a HTML 5 fueron:

- | Basarse en HTML, CSS, DOM y Javascript.
- | Reducir la necesidad del uso de plugins, como por ejemplo flash.
- | Mejorar el tratamiento de errores.
- | Crear etiquetas que reemplacen el uso de scripts.
- | Lenguaje que pueda utilizarse en cualquier tipo de dispositivo, como móviles, PDA's, etc.



Autoevaluación

- 1| Investiga sobre el funcionamiento de HTML5 en los diferentes navegadores
- 2| Detalla como es el perfil de un webmaster y un desarrollador web
- 3| Investiga como es el mercado laboral para un desarrollador web y que lenguajes se utilizan en las empresas actuales

Créditos

Imágenes

Encabezado: Image by Steve Buissinne from Pixabay

<https://pixabay.com/photos/pawn-chess-pieces-strategy-chess-2430046/>

Tipografía

Para este diseño se utilizó la tipografía *Source Sans Pro* diseñada por Paul D. Hunt. y *Space Mono* diseñada por Colophon Foundry Extraída de Google Fonts.

Si detectás un error del tipo que fuere (falta un punto, un acento, una palabra mal escrita, un error en código, etc.), por favor comunicate con nosotros a correcciones@issd.edu.ar e indicanos por cada error que detectes la página y el párrafo. Muchas gracias por tu aporte.